SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO2: Design UML diagrams to represent system requirements and architecture. *(BL3)*

Assessment Tool Weightage: 30%

QUESTIONS: 6 **Total Marks:30**

Annexure A

Scenario-Based

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Judy Allen J

1. Use Case Diagram Scenario

A Smart Pharmacy Management System enables customers to search medicines, upload prescriptions, place medicine orders, and make online payments. Pharmacists verify prescriptions and dispense medicines, while administrators manage medicine inventory and user accounts. The interactions among system users are not clearly defined.

Based on this scenario, analyze the system requirements, identify all actors and use cases, and design a Use Case Diagram showing appropriate relationships **(**include, extend, and generalization) to ensure complete system functionality.

Answer:

Scenario Analysis
The Smart Pharmacy Management System allows customers to search medicines, upload prescriptions, place medicine orders, and make online payments. Pharmacists verify prescriptions and dispense medicines, while administrators manage medicine inventory and user accounts. The system requires proper identification of actors, use cases, and their relationships to ensure complete functionality.

Actors
Primary Actors
- Customer
- Pharmacist
- Administrator

Secondary Actor
- Payment Gateway

Use Cases
Customer
- Register/Login
- Search Medicines
- Upload Prescription
- Place Order
- Make Online Payment
- Track Order
- View Order History

Pharmacist
- Verify Prescription
- Approve/Reject Prescription
- Dispense Medicines
- Update Order Status

Administrator
- Manage Medicine Inventory
- Add/Update/Delete Medicines
- Manage User Accounts
- Generate Reports

Payment Gateway
- Process Payment

Relationships

<<include>> Relationships

- Place Order <<include>> Search Medicines
- Place Order <<include>> Upload Prescription (only for prescription medicines)
- Make Online Payment <<include>> Process Payment
- Manage Inventory <<include>> Add Medicine
- Manage Inventory <<include>> Update Medicine
- Manage Inventory <<include>> Delete Medicine

<<extend>> Relationships

- Verify Prescription <<extend>> Place Order
- Reject Prescription <<extend>> Verify Prescription
- Approve Prescription <<extend>> Verify Prescription
- Track Order <<extend>> Place Order

Generalization

User

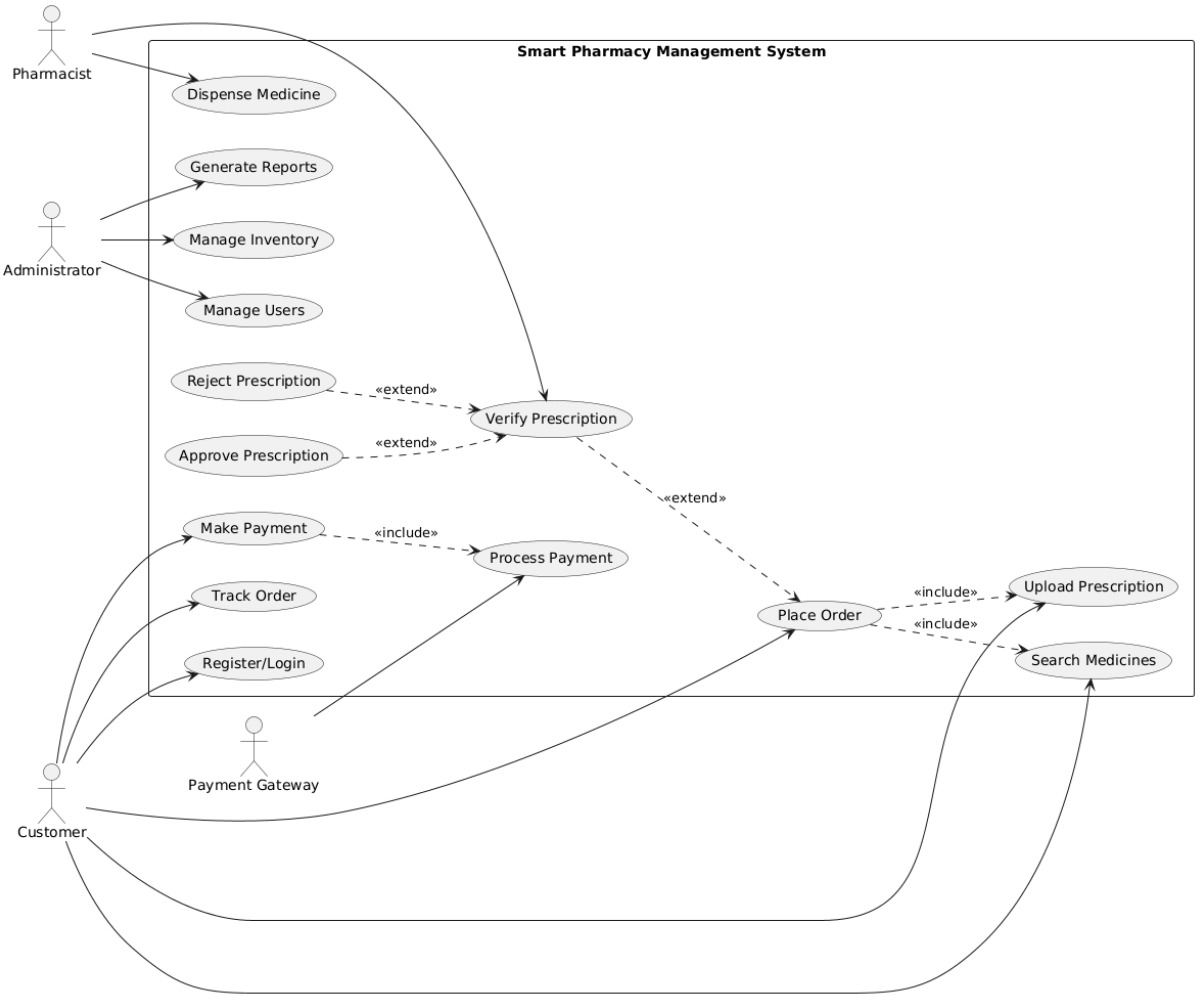
- Customer
- Pharmacist
- Administrator

All three actors inherit common functions such as Login and Logout.

Decision and Justification

The Use Case Diagram should contain three primary actors (Customer, Pharmacist, and Administrator) and one secondary actor (Payment Gateway). Include relationships are used for mandatory operations such as processing payments and managing inventory, while Extend relationships are used for optional or conditional activities such as prescription verification and order tracking. Generalization is applied because all users share common authentication functions.

This design clearly represents the interactions among all users, avoids redundancy, and improves system modularity and maintainability.


Smart Pharmacy Management System
Pharmacist
Administrator
Customer
Payment Gateway
Dispense Medicine
Generate Reports
Manage Inventory
Manage Users
Reject Prescription
Approve Prescription
Verify Prescription
Make Payment
Process Payment
Track Order
Register/Login
Place Order
Upload Prescription
Search Medicines
«extend»
«extend»
«extend»
«include»
«include»
«include»

2. Class Diagram Scenario

A Smart Hotel Management System consists of entities such as Guest, Room, Reservation, Payment, and Receptionist. The existing design does not clearly define class attributes, operations, or relationships among these entities

Based on this scenario, identify the required classes, define suitable attributes and methods, establish relationships **(**association, aggregation, composition, and inheritance), specify multiplicity, and construct a detailed Class Diagram**.**

Answer:

Scenario Analysis

The Smart Hotel Management System includes the entities Guest, Room, Reservation, Payment, and Receptionist. The existing design does not clearly specify class attributes, methods, relationships, or multiplicity. A proper Class Diagram should identify these elements and show how they interact using association, aggregation, composition, and inheritance.

Classes with Attributes and Methods

1. Guest

Attributes

- guestID
- name
- phoneNumber
- email
- address

Methods

- bookRoom()
- cancelReservation()
- makePayment()
- viewReservation()

2. Room

Attributes

- roomNumber
- roomType
- price
- availability

Methods

- checkAvailability()
- updateStatus()

3. Reservation

Attributes

- reservationID
- checkInDate
- checkOutDate
- reservationStatus

Methods

- createReservation()
- cancelReservation()
- calculateBill()

4. Payment

Attributes

- paymentID
- amount
- paymentMode
- paymentStatus

Methods

- processPayment()
- generateReceipt()

5. Receptionist

Attributes

- employeeID
- name
- contactNumber

Methods

- checkInGuest()
- checkOutGuest()
- assignRoom()

Relationships

Association

- A Guest books a Reservation.
- A Receptionist manages Reservations.

Aggregation

- A Room can exist independently even if there are no reservations.
- Therefore,
    - **Reservation ◇── Room**

Composition

- A Reservation owns a Payment.
- If a reservation is deleted, its payment record is also removed.
- Therefore,
    - **Reservation ◆── Payment**

Inheritance

Create a common superclass:

Person

- personID
- name
- phoneNumber

Methods

- login()
- logout()

Subclasses:

- Guest
- Receptionist

Multiplicity

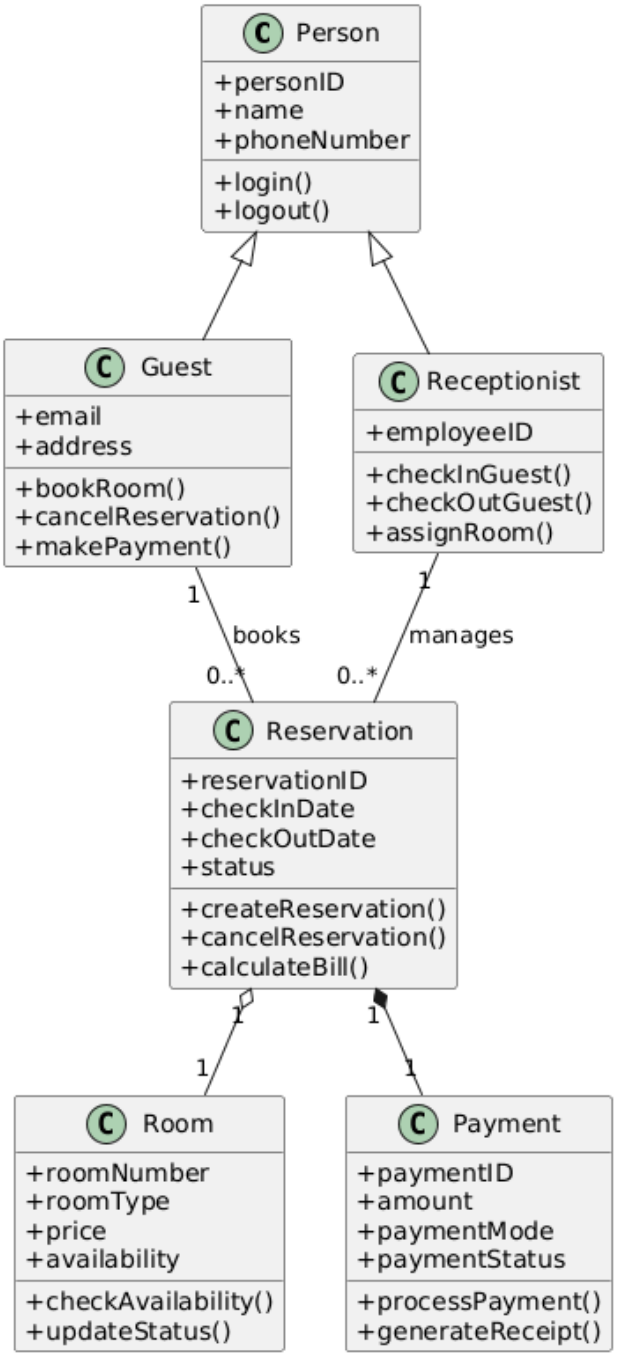
Relationship	Multiplicity
Guest → Reservation	1 Guest can have 0..* Reservations
Reservation → Room	1 Reservation is for 1 Room
Room → Reservation	1 Room can have many Reservations (at different times)
Reservation → Payment	1 Reservation has exactly 1 Payment
Receptionist → Reservation	1 Receptionist manages many Reservations

Decision and Justification

The proposed Class Diagram clearly separates the responsibilities of each class.

- Inheritance removes duplicate information by introducing the Person superclass.
- Association models interactions between guests, receptionists, and reservations.
- Aggregation is used because rooms exist independently of reservations.
- Composition is used because a payment cannot exist without its reservation.
- Proper multiplicity ensures that relationships between classes are accurately represented.

This design improves modularity, maintainability, scalability, and code reusability, making the Smart Hotel Management System easy to expand with future features such as online booking, loyalty programs, and room service.



3. Sequence Diagram Scenario

In an Online Banking System, a customer logs in, transfers funds, verifies the transaction using OTP, and receives a transaction confirmation. The communication among the customer, authentication service, banking server, and database is incomplete.

Based on this scenario, analyze the sequence of operations, identify participating objects and message flows, and design a Sequence Diagram representing the complete interaction process.

Answer:

Scenario Analysis

The Online Banking System allows a customer to log in, transfer funds, verify the transaction using an OTP, and receive a confirmation message. The Sequence Diagram should clearly show the interaction between the Customer, Authentication Service, Banking Server, and Database in chronological order.

Participating Objects

- Customer
- Authentication Service
- Banking Server
- Database

Sequence of Operations

1. Customer enters login credentials.
2. Authentication Service validates the credentials.
3. Database verifies the customer details.
4. Login is successful.
5. Customer requests fund transfer.
6. Banking Server checks account balance.
7. Database returns account details.
8. Banking Server generates an OTP.
9. Customer enters the OTP.
10. Authentication Service verifies the OTP.
11. Banking Server transfers the amount.
12. Database updates both account balances.
13. Banking Server sends transaction confirmation.
14. Customer receives the success message.

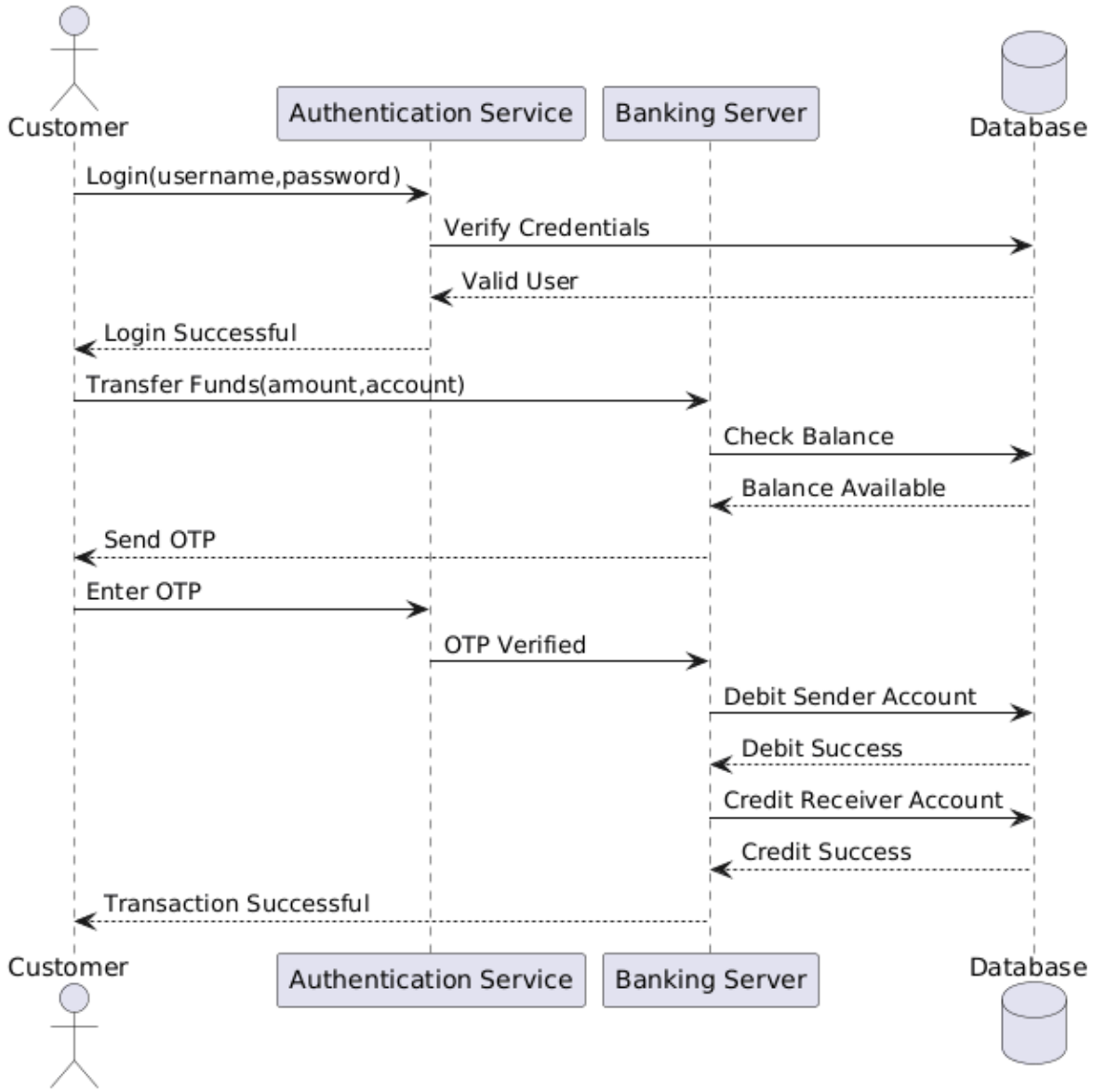
Decision and Justification

The Sequence Diagram follows the exact order in which events occur during an online banking transaction.

- Authentication is completed before allowing fund transfer.
- OTP verification provides additional security.
- The database updates account balances only after successful OTP verification.

- A confirmation message is sent only after the transaction is completed successfully.

This sequence ensures security, accuracy, reliability, and proper transaction processing in the Online Banking System.



4. Activity Diagram Scenario

A Passport Application System allows applicants to submit applications, upload supporting documents, complete identity verification, pay processing fees, and receive passports. The workflow does not clearly represent approval, rejection, or concurrent activities.

Based on this scenario, analyze the workflow, identify decision points and parallel activities, and design an optimized Activity Diagram using decision nodes, forks, and joins.

Answer:

Scenario Analysis

The Passport Application System allows applicants to submit passport applications, upload supporting documents, complete identity verification, pay processing fees, and receive passports. The workflow should include approval and rejection decisions as well as parallel activities using fork and join nodes.

Workflow Analysis

Main Activities

- Submit Passport Application
- Upload Required Documents
- Identity Verification
- Document Verification
- Application Review
- Pay Processing Fee
- Passport Printing
- Passport Dispatch
- Receive Passport

Decision Points

Decision 1

Are all required documents uploaded?

- Yes → Continue
- No → Reject Application

Decision 2

Is identity verification successful?

- Yes → Continue
- No → Reject Application

Decision 3

Is fee payment successful?

- Yes → Print Passport
- No → Cancel Application

Parallel Activities (Fork and Join)

After the application is submitted:

Two activities occur simultaneously:

- Upload Documents
- Identity Verification

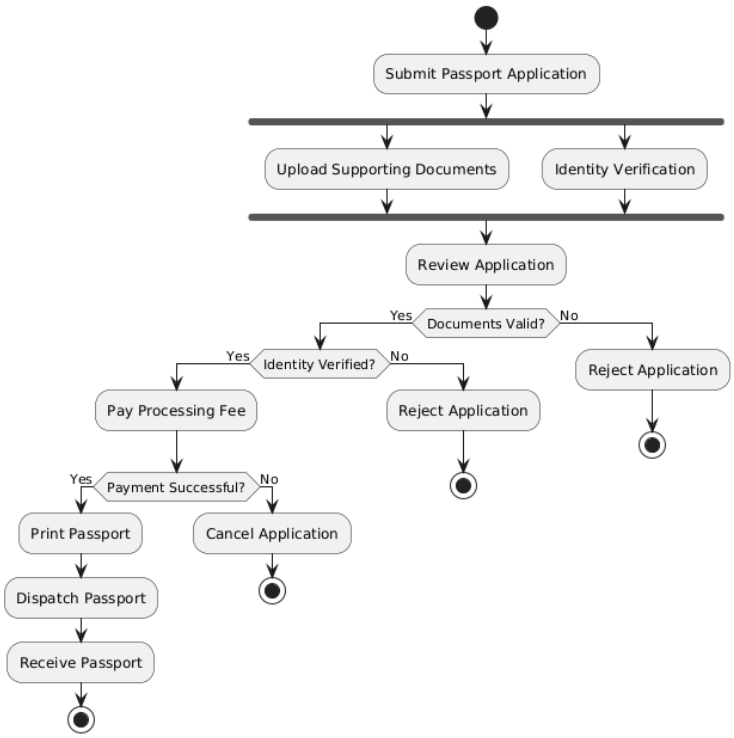
These two activities are synchronized using a Join Node before the application review begins.

Decision and Justification

The Activity Diagram begins with the applicant submitting the application. The system performs document upload and identity verification simultaneously using a Fork Node, reducing processing time. Both activities must be completed before proceeding to application review using a Join Node.

Decision nodes determine whether the application is approved or rejected based on document verification, identity verification, and payment status. Only successful applications proceed to passport printing and dispatch.

This workflow improves efficiency, accuracy, transparency, and faster passport processing while reducing delays.



5. State Diagram Scenario

A Smart Elevator Control System operates through states such as Idle, Door Open, Door Closed, Moving Up, Moving Down, Emergency Stop, and Maintenance Mode. The transitions between these states are not clearly specified.

Based on this scenario, identify all possible states, triggering events, and transitions, and construct a State Transition Diagram that accurately represents the elevator's behavior.

Answer:

Scenario Analysis

The Smart Elevator Control System operates through the states Idle, Door Open, Door Closed, Moving Up, Moving Down, Emergency Stop, and Maintenance Mode. The State Diagram should identify all possible states, the events that trigger transitions, and the movement between states to accurately represent the elevator's behavior.

States

- Initial State
- Idle
- Door Open
- Door Closed
- Moving Up
- Moving Down
- Emergency Stop
- Maintenance Mode
- Final State

Triggering Events

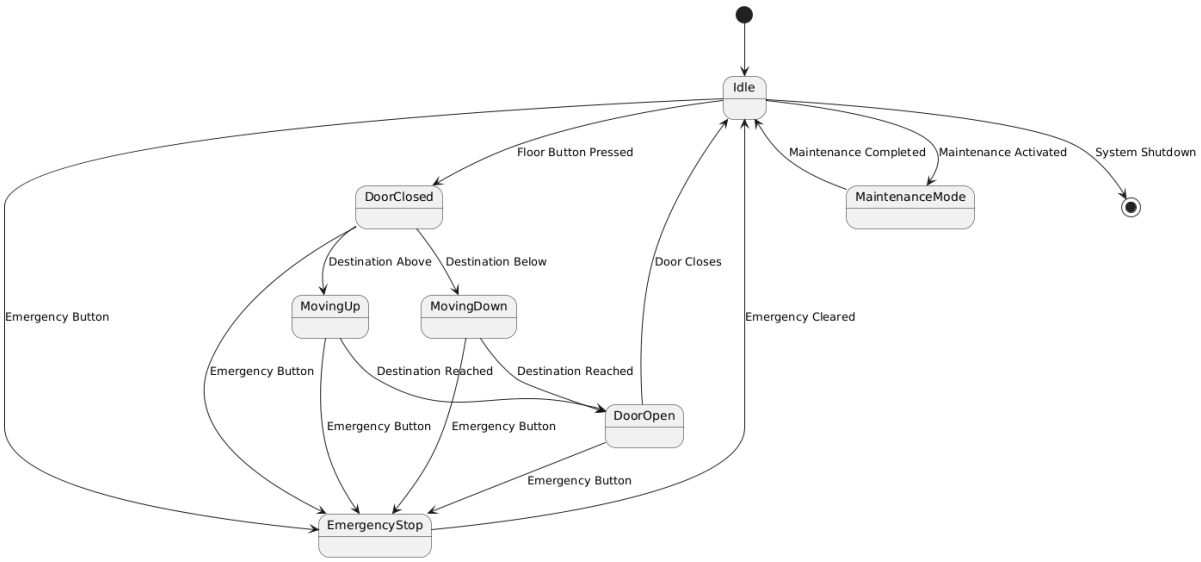
Current State	Event	Next State
Initial	Elevator Ready	Idle
Idle	Floor Button Pressed	Door Closed
Door Closed	Destination Above	Moving Up
Door Closed	Destination Below	Moving Down
Moving Up	Destination Reached	Door Open
Moving Down	Destination Reached	Door Open
Door Open	Door Timer Expires	Idle
Any State	Emergency Button Pressed	Emergency Stop
Emergency Stop	Emergency Cleared	Idle
Idle	Maintenance Activated	Maintenance Mode
Maintenance Mode	Maintenance Completed	Idle

Decision and Justification

The elevator starts in the Idle state, waiting for a user request. When a floor button is pressed, the doors close before movement begins. Depending on the requested floor, the elevator transitions to either Moving Up or Moving Down. Once the destination is reached, the doors open to allow passengers to enter or exit. After a short delay, the doors close and the elevator returns to the Idle state.

If the emergency button is pressed at any time, the elevator immediately enters the Emergency Stop state. After the emergency is resolved, normal operation resumes. The system can also enter Maintenance Mode when maintenance is required, preventing normal operation until maintenance is completed.

This design ensures safe, reliable, and efficient elevator operation**.**



6. Deployment Diagram Scenario

A Smart Home Automation System uses smart sensors, IoT controllers, a home gateway, cloud servers, and a mobile application to monitor and control lighting, security, and appliances remotely. The deployment architecture does not clearly illustrate hardware nodes, deployed software, or communication channels.

Based on this scenario, analyze the deployment environment, identify all hardware nodes and software artifacts, and design a Deployment Diagram showing the physical architecture and communication links.

Answer:

Scenario Analysis

The Smart Home Automation System uses smart sensors, IoT controllers, a home gateway, cloud servers, and a mobile application to monitor and control lighting, security systems, and household appliances remotely. The Deployment Diagram should clearly identify the hardware nodes, deployed software artifacts, and communication links between them.

Hardware Nodes

- Mobile Device
- Home Gateway
- IoT Controller
- Smart Sensors
- Cloud Server

Software Artifacts

Mobile Device

- Smart Home Mobile App

Home Gateway

- Gateway Software
- Communication Manager

IoT Controller

- Device Control Software
- Automation Engine

Cloud Server

- Cloud Database
- User Authentication Service
- Monitoring Service
- Notification Service

Smart Sensors

- Sensor Firmware

Communication Links

- Mobile App ↔ Cloud Server (Internet)
- Cloud Server ↔ Home Gateway (Secure Internet)
- Home Gateway ↔ IoT Controller (Wi-Fi/LAN)
- IoT Controller ↔ Smart Sensors (ZigBee/Bluetooth/Wi-Fi)

Decision and Justification

The deployment architecture places the Mobile Application on the user's smartphone, allowing remote monitoring and control of home devices. Commands are sent securely to the Cloud Server, which authenticates users and stores sensor data.

The Home Gateway acts as a bridge between the cloud and local smart devices. The IoT Controller manages lighting, security systems, and appliances by communicating with the Smart Sensors.

This architecture provides:

- Secure remote access
- Real-time monitoring
- Efficient communication
- Easy scalability
- Reliable device management

Therefore, the Deployment Diagram accurately represents the physical architecture and communication channels of the Smart Home Automation System.